Information Processing Letters 109 (2009) 975–979

Contents lists available at ScienceDirect

Information Processing Letters

www.elsevier.com/locate/ipl

An optimal algorithm for the maximum-density path in a tree

Bang Ye Wu *

Dept. of Computer Science and Information Engineering, National Chung Cheng University, Taiwan

ARTICLE INFO

Article history:
Received 22 May 2008
Received in revised form 14 April 2009
Available online 13 May 2009
Communicated by W.-L. Hsu

Keywords:
Algorithms
Tree
Optimization problem
Spine decomposition

ABSTRACT

We studied the problem of finding the maximum-density path in a tree. By spine decomposition and a linear-time algorithm for the maximum density segment problem, we developed an $O(n \log n)$ time algorithm, which improves the previously best result of $O(n \log^2 n)$ by using centroid decomposition. We also show the time complexity is optimal in the algebraic computation tree model.

1. Introduction

We investigated the following problem of finding the maximum-density path in a tree.

> PROBLEM: Maximum-Density Path in a Tree (TreeMDP)
> INSTANCE: An n-node tree $T = (V, E)$, a weight w_e and a positive length l_e for each edge $e \in E$, and two constant values $L_{\min} \leqslant L_{\max}$.
> GOAL: Find a simple path P in T such that its density $w(P)/l(P)$ is maximum among all paths of lengths within the interval $[L_{\min}, L_{\max}]$, in which $l(P) = \sum_{e \in P} l_e$ and $w(P) = \sum_{e \in P} w_e$ are the length and the weight of the path P, respectively.

The problem is a generalization of the length-constrained maximum-density segment problem which can be regarded as finding a subpath in a path. Because of applications to biological sequence analysis, the maximum-density segment problem has attracted many researchers and has some variants. For the case that all edges have uniform length, i.e., $l_e = 1$ for all edge e, Lin, Jiang, and Chao [10] gave an $O(n \log L_{\min})$-time algorithm, and the time complexity was improved to $O(n)$-time by Goldwasser, Kao, and Lu [6]. A linear time algorithm of the more general case that edge lengths are not necessarily uniform was given by Chung and Lu [4]. More references can also be found in their paper.

On the other hand, Wu et al. [13] studied the problem of finding a length-constrained heaviest path in a tree and gave an $O(n \log^2 n)$-time algorithm. Recently the time complexity is improved to $O(n \log n)$ by Liu and Chao [11]. For the maximum-density path problem in trees, Lin et al. [9] proposed an $O(nL_{\min})$-time algorithm for the uniform case. For a more general case that there are constraints on both the path weights and lengths, another pseudo-polynomial time algorithm was proposed in [7]. Since there are totally $O(n^2)$ simple paths in a tree, a naive algorithm takes $O(L_{\max} n^2)$ time, and the time complexity can be easily reduced to $O(n^2)$. Lin's algorithm is efficient only when the length constraint $L_{\min}$ is small. Lau et al. developed an $O(n \log^2 n)$ time algorithm by using centroid decomposition [8]. In this paper, we improve the time complexity to $O(n \log n)$ by using spine decomposition. Our algorithm uses the divide-and-conquer strategy. The dividing step is based on spine decomposition and the merging step employs Chung and Lu's linear time algorithm for the maximum density segment problem [4]. By reduction from the Set Intersection problem, we show that the time complexity of our algorithm is optimal.

* Correspondence address: National Chung Cheng University, ChiaYi, Taiwan 621, ROC.
E-mail address: bangye@cs.ccu.edu.tw.

0020-0190/$ – see front matter
doi:10.1016/j.ipl.2009.05.005

The rest of the paper is organized as follows. In Section 2, we show some preliminaries including the lower bound of the time complexity and a linear time algorithm for a simplified version of the problem. In Section 3, we first show an $O(n \log^2 n)$ time algorithm by centroid decomposition, and then improve the time complexity to $O(n \log n)$ by spine decomposition. Finally we give some concluding remarks in Section 4.

2. Preliminaries and the lower bound

A *twin star* is a tree with two internal nodes. The TreeMDP problem will be named as TwinMDP if the input tree is restricted to a twin star. We shall show that even for this special case the time complexity of the problem has an $\Omega(n \log n)$ lower bound. The idea came from [11]. Consider the following problem.

PROBLEM: Set Intersection Problem
INSTANCE: Two sets $\{x_1, x_2, \ldots, x_n\}$ and $\{y_1, y_2, \ldots, y_n\}$ of positive integers.
GOAL: Determine whether there exist indices i and j such that $x_i = y_j$.

It was shown that the Set Intersection Problem has an $\Omega(n \log n)$ lower bound of time complexity in the algebraic computation tree model [1].

Theorem 1. *Even when the input tree is restricted to a twin star, the TreeMDP problem has an $\Omega(n \log n)$ lower bound in the algebraic computation tree model.*

Proof. We shall show that the Set Intersection problem can be transformed to the TwinMDP problem in linear time. Given two sets $\{x_1, x_2, \ldots, x_n\}$ and $\{y_1, y_2, \ldots, y_n\}$ of positive integers, we construct a twin star as follows. Let c_1 and c_2 be the two center nodes, and $l(c_1, c_2) = w(c_1, c_2) = 1$. We determine a number L larger than all numbers x_i and y_i in linear time, and create n leaf edges e_i, $1 \leqslant i \leqslant n$, incident to c_1. Similarly we create n leaf edges f_i, $1 \leqslant i \leqslant n$, incident to c_2. The lengths and weights are set by $l(e_i) = w(e_i) = L + x_i$ and $l(f_i) = w(f_i) = L - y_i$ for all i. Finally let $L_{\min} = L_{\max} = 2L + 1$.

It can be verified that there exists $x_i = y_j$ if and only if there is a path of length $2L + 1$. Therefore the TwinMDP problem has an $\Omega(n \log n)$ lower bound in the algebraic computation tree model. Since TwinMDP is a special case of TreeMDP, the lower bound also holds for the TreeMDP problem. □

Although the TwinMDP problem has an $\Omega(n \log n)$ lower bound, we now show the problem can be solved in linear time if the edges are already sorted by their lengths. Our algorithm for the TreeMDP problem uses it at the merging step. Let us first take a look at the following problem.

Given a sequence $\{(w_i, l_i)\}_{i=1}^n$ with $l_i \geqslant 0$ and two numbers $L_{\min} \leqslant L_{\max}$, the maximum-density segment (MDS) problem looks for two indices $j \leqslant k$ such that $\sum_{i=j}^k l_i$ is in the interval $[L_{\min}, L_{\max}]$ and $(\sum_{i=j}^k w_i)/(\sum_{i=j}^k l_i)$ is maximum. Chung and Lu [4] gave a linear time algorithm for the problem. They also showed that the following variant can also be solved in linear time: Besides the above input data, we are given two "range-constrained" intervals $[x_1, y_1]$ and $[x_2, y_2]$, $y_1 \leqslant x_2$, such that the solution is restricted to $j \in [x_1, y_1]$ and $k \in [x_2, y_2]$. We shall call this variant the *RangeMDS* problem. Particularly, when $x_1 = 1$, $y_1 = x_2$ and $y_2 = n$, the RangeMDS problem asks for the best interval containing a given index y_1.

Now consider the *sorted TwinMDP* problem defined as follows. The definition is the same as TwinMDP except that

(1) the solution must contain the center edge; and
(2) the weights and the lengths of the leaf edges are given by two sequences $\{(W_i, L_i)\}_{i=1}^n$ and $\{(W'_i, L'_i)\}_{i=1}^m$, in which $0 < L_1 < L_2 < \cdots < L_n$ are lengths of leaf edges incident to one center node and $0 < L'_1 < L'_2 < \cdots < L'_m$ are lengths of those to the other center node.

When there is no confusion, we shall simply say "the best path" instead of "the maximum-density path satisfying the length constraints". Any feasible solution of the sorted TwinMDP problem is a 3-edge path containing the center edge. In other words, it looks for an index pair (j, k) such that $L_j + L_0 + L'_k$ satisfies the length constraint and $(W_j + W_0 + W'_k)/(L_j + L_0 + L'_k)$ is maximized, in which L_0 and W_0 are the length and the weight of the center edge. We can easily transform the sorted TwinMDP problem into the RangeMDS problem as follows. We set $w_n = W_1$; $w_{n-i} = W_{i+1} - W_i$ for $1 \leqslant i \leqslant n-1$; $w_{n+1} = W_0$; $w_{n+2} = W'_1$; and $w_{n+2+i} = W'_{i+1} - W'_i$ for $1 \leqslant i \leqslant m-1$. The values of l_i's are assigned similarly. The two range-constrained intervals are given by $[1, n+1]$ and $[n+1, n+m+1]$, and the length constraint remains the same. It can be verified that the transformation takes only linear time and all the feasible solutions of the sorted TwinMDP problem are 1-to-1 mapping to the ones of the RangeMDS problem. Therefore the sorted TwinMDP problem can be solved in linear time by Chung and Lu's algorithm [4]. We state the result as the following corollary.

Corollary 2. *The sorted TwinMDP problem can be solved in linear time.*

An unrooted tree is binary if all the internal nodes have degree at most 3. In the next lemma we show a conversion of arbitrary tree to binary tree, which was also previous used in [12].

Lemma 3. *If there exists an $O(n \log n)$ time algorithm for the TreeMDP problem on binary trees, the problem on arbitrary trees can also be solved with the same time complexity, in which n is the number of nodes.*

Proof. For an arbitrary input tree T, we transform it into a binary tree T' by inserting some edges of zero-length and zero-weight. Let Y be the set of the simple paths in T' such that neither the first nor the last edge is of zero-length. The lemma can be easily obtained by the following two simple observations. First there is a 1-to-1 correspondence between the simple paths in T and the paths in Y. Secondly, the number of nodes in T' is at most $2n - 3$. □

In the remaining paragraphs, we assume that the input $T = (V, E)$ is an n-node unrooted binary tree. An unrooted tree can be rooted at any of its node in linear time. For a rooted tree T and a node v, let T_v denote the subtree rooted at v. Let v be a child of r. The *branch* of T_r at v consists of the subtree T_v and the edge (r, v). The best path within a branch of T_r at v will be denoted by $\mathrm{MDP}(T_r, v)$.

3. The algorithms

A centroid of a tree is a node whose removal divides the tree into subtrees of at most a half of the nodes. A tree has one or two centroids and a centroid can be found in linear time. Centroid decomposition has been used to solve problems on trees, for example [5,8,13]. By Corollary 2 and centroid decomposition, we can show that the TreeMDP problem can be solved in $O(n \log^2 n)$ time as follows. We first find a centroid to divide the tree into subtrees and then recursively find the best path in each subtree. At a merging step, we check the paths with two endpoints in two different subtrees. The reason to use centroid decomposition is that the number of nodes in each subtree is at most one half and therefore the recursion depth is bounded by $O(\log n)$. The following is such an algorithm, which is similar to the one in [8].

Algorithm A1.

1: if the tree contains only one node then
   return $-\infty$;
2: root T at its centroid r;
3: for each child v of r do
   find $\mathrm{MDP}(T_r, v)$ recursively;
4: choose the best path P_1 among the paths at Step 3.
5: for each child v of r do
   find the weights and lengths of the paths from r to all nodes in T_v;
6: sort the pairs of weights and lengths by their lengths;
7: for each pair u and v of children of r do
   find the best path containing path (u, r, v) by solving a sorted TwinMDP problem;
8: choose the best path P_2 among the paths at Step 7;
9: return the better of P_1 and P_2.

Lemma 4. *The Algorithm* A1 *solves the TreeMDP problem in* $O(n \log^2 n)$ *time.*

Proof. The correctness can be easily verified since a path is either entirely within one of the branches or across the centroid. The time complexity of each recursion is dominated by Step 6 which takes $O(n \log n)$ time for an input tree of n nodes. Note that Step 7 takes only $O(n)$ time by Corollary 2 and by the fact that r has at most 3 children since the tree is binary. Let n_i denote the number of nodes in the subtrees. The time complexity $t(n)$ of the algorithm can be formulated by the following recursive relation.

$$t(n) = \sum_{i=1}^{3} t(n_i + 1) + O(n \log n),$$

in which $n_i \leqslant n/2$ and $\sum_{i=1}^{3} n_i = n - 1$. We have $t(n) \in O(n \log^2 n)$. □

By spine decomposition, we shall improve the time complexity to $O(n \log n)$. Our idea of spine decomposition came from [3]. But for our convenience, we describe it in a way not exactly the same as theirs.

A rooted binary tree is *right-skew* if, for any internal node, the number of nodes in its right subtree is more than or equal to the one in the left. In linear time we can make a binary tree right-skew. For such a tree, the path from the root to the right-most leaf is defined as the *spine* of the tree. The spine decomposition of a tree is a process of removing the spine and recursively decomposing the remaining subtrees. We define the level of a spines as follows. The level of the spine of the input tree is one. A spine of a subtree resulted by removing a spine of level i is of level $i + 1$. The maximum level of the spines is defined as the level of the spine decomposition. Let S be a spine and v a node on S. The left branch of v, denoted by $B(S, v)$, consists of its left subtree and the edge between v and its left child. Since the tree is right-skew, every subtree resulted by removing the spine has at most a half of the nodes. Consequently a left branch of a spine of level i has at most $n/2^i$ nodes. Furthermore the level of the spine decomposition is $O(\log n)$.

For the spine S of a tree or subtree, we build a balanced merge tree $\mathrm{BMT}(S)$ as follows.

(1) Create a root for $\mathrm{BMT}(S)$.
(2) If the spine contains only one node, the process is finished. Otherwise, we divide the spine into two subpaths S_1 and S_2 such that the numbers of nodes in the left branches of the two sub-spines are as equal as possible. Precisely speaking, if $S = (v_1, v_2, \ldots, v_k)$ and n_i is the number of nodes in $B(S, v_i)$, then $S_1 = (v_1, v_2, \ldots, v_j)$ such that $\sum_{i=1}^{j} n_i \leqslant n/2$ and $\sum_{i=1}^{j+1} n_i > n/2$, in which $n = \sum_{i=1}^{k} n_i$.
(3) Recursively build $\mathrm{BMT}(S_1)$ and $\mathrm{BMT}(S_2)$ and make the roots of them as the two children of the root of $\mathrm{BMT}(S)$.

The following property comes from [2,3], which shows that a BMT can be constructed in linear time.

Property 5. *The BMT of a spine of k nodes can be constructed in* $O(k)$ *time if the number of nodes in each left branch is given.*

A node q of $\mathrm{BMT}(S)$ corresponds to a subpath S' of S. The two endpoints of S' are denoted by $\lambda_1(q)$ and $\lambda_2(q)$, in which $\lambda_1(q)$ is the node closer to the root of the tree T. Particularly if q is a leaf of $\mathrm{BMT}(S)$, then $\lambda_1(q) = \lambda_2(q)$ is the spine node corresponding to q. The node set of S' and all subtrees left to S' is denoted by $C(q)$, and we say that the nodes are covered by q. Let v be a node of spine S and q the corresponding node (a leaf) in the BMT. It can be shown by induction that the depth of q, i.e., the number of BMT edges from q to the root s, is

$$\left\lceil \log\left(\left|C(s)\right|\right) / \left|B(S, v)\right| \right\rceil, \tag{1}$$

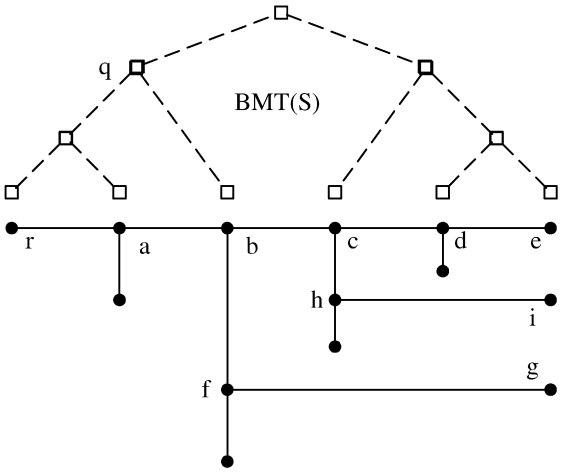

Fig. 1. The spine decomposition of a binary right-skew tree: the tree is drawn in solid line, and r is the root. The right and the left children of c are d and h, respectively. $S = (r, a, b, c, d, e)$ is the spine of level 1, and (c, h, i) and (b, f, g) are spines of level 2. The balanced merge tree (BMT) of S is drawn by dash line and nodes of BMT(S) are shown as squares. The depth of q is 1, $\lambda_1(q) = r$, $\lambda_2(q) = b$, and $C(q)$ contains 7 nodes of the tree. Note that the BMT is built in a balanced way.

in which $|C(s)|$ and $|B(S, v)|$ are the numbers of nodes in $C(s)$ and $B(S, v)$, respectively. A spine decomposition is illustrated in Fig. 1.

In our algorithm, each node q of BMT(S) is associated with a data structure $D(q)$ as follows. Recall that the length, and weight respectively, of a path is the sum of the length, and weight respectively, of individual edges.

- $q.best$: the best solution entirely within $C(q)$.
- $q.LList$: the length-sorted list (W_i, L_i) of weights and lengths of paths from $\lambda_1(q)$ to all nodes covered by q.
- $q.RList$: the length-sorted list (W_i, L_i) of weights and lengths of paths from $\lambda_2(q)$ to all nodes covered by q.

Lemma 6. *Let q be a node of BMT(S). Then $D(q)$ can be computed from $D(q_1)$ and $D(q_2)$ in $O(|C(q)|)$ time, in which q_1 and q_2 are the left and the right children of q.*

Proof. To compute $q.LList$ from its children, we append the path between $\lambda_1(q_1)$ and $\lambda_1(q_2)$ to all paths in $q_2.LList$, and then merge it with $q_1.LList$. $q.RList$ can be computed similarly. The value $q.best$ is the maximum of $q_1.best$, $q_2.best$, and the best path with one endpoint in $C(q_1)$ and the other in $C(q_2)$, which can be computed from $q_1.RList$ and $q_2.LList$ by solving a sorted TwinMDP problem in linear time (Corollary 2). The total time complexity of merging two children of a node q is linear in $|C(q)|$. □

Our algorithm for the TreeMDP problem is listed in the following.

Algorithm A2.

1: transform T into a binary tree T';
2: transform T' into a right-skew tree T'';
3: build the BMT of the spine S of T''; let r be the root;

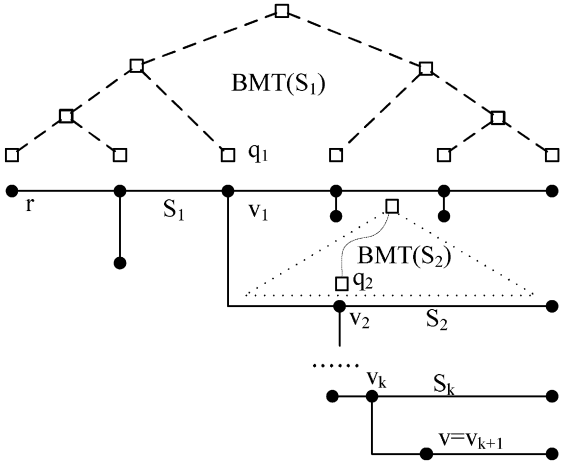

Fig. 2. Spine structure.

4: call subroutine MergeSpine(r) and output the returned solution;

Subroutine MergeSpine(r)

5: if r is not a leaf in the BMT then
   let q_1 and q_2 are the children of r;
6: call MergeSpine(q_1) and MergeSpine(q_1);
7: compute $D(r)$ from $D(q_1)$ and $D(q_2)$;
   return $D(r)$;
   else
8: let v be the spine node corresponding to r;
9: if v has no left branch then
10: set $r.best$ to $-\infty$;
    set both $r.LList$ and $r.RList$ to Null;
    else
11: build BMT(Y) of the spine Y of the left branch of v;
12: call MergeSpine(q), in which q is the root of BMT(Y);
13: set $r.best$ to $q.best$; set both $r.LList$ and $r.RList$ to $q.LList$;
14: return $D(r)$.

Theorem 7. *Algorithm* A2 *solves the TreeMDP problem in $O(n \log n)$ time which is optimal in the algebraic computation tree model.*

Proof. The correctness is similar to Algorithm A1 and can be easily verified. We shall focus on the time complexity. Steps 1 and 2 of the main program takes obviously linear time. What we need to analyze is the time for building those BMTs (Steps 3 and 11) and the time for computing $D(r)$ from the two children (Step 7).

First, by Property 4, the BMT of a k-node spine can be built in $O(k)$ time. Since each node is involved in at most two spines, the total time complexity for constructing all the BMTs is $O(n)$. Second, the total time complexity of Step 7 in all the recursive calls is bounded by $O(\sum_r |r.LList|)$, in which $|r.LList|$ is the number of elements in the list and the summation is taken over all

nodes in all BMTs. In another point of view, $\sum_r |r.LList| = \sum_{v \in V} |\{r | v \in C(r)\}|$. We shall show that every node v is covered by at most $O(\log n)$ BMT nodes in total, and then the result follows.

For any node $v \in V$, we observe the structure of the spines it belongs to. Suppose that $v = v_{k+1} \in B(S_k, v_k)$, $v_k \in B(S_{k-1}, v_{k-1})$, $\ldots$, $v_2 \in B(S_1, v_1)$, in which S_i is a spine of level i. The structure is illustrated in Fig. 2. Let q_i be the node corresponding to v_i in BMT(S_i). By Eq. (1), the depth of q_1 is $\lceil \log(n/|B(S_1, v_1)|) \rceil$, and for $2 \leqslant i \leqslant k$, the depth of q_i is $\lceil \log(\frac{|B(S_{i-1}, v_{i-1})|}{|B(S_i, v_i)|}) \rceil$. Therefore, $\sum_{v \in V} |\{r | v \in C(r)\}|$ is the sum of the depths of q_i and can be calculated by

$$\begin{aligned}
&\left\lceil \log\left(\frac{n}{|B(S_1, v_1)|}\right)\right\rceil + \sum_{i=2}^{k} \left\lceil \log\left(\frac{|B(S_{i-1}, v_{i-1})|}{|B(S_i, v_i)|}\right)\right\rceil \\
&\quad \leqslant k + \log\left(\frac{n}{|B(S_1, v_1)|}\right) + \sum_{i=2}^{k} \log\left(\frac{|B(S_{i-1}, v_{i-1})|}{|B(S_i, v_i)|}\right) \\
&\quad = k + \log\left(\frac{n}{|B(S_k, v_k)|}\right) \\
&\quad \leqslant k + \log n \leqslant 2 \log n.
\end{aligned}$$

The last inequality is obtained by that k is bounded by the level of spine decomposition and therefore at most $\log n$. As a result, each node is processed $O(\log n)$ times and the time complexity is bounded by $O(n \log n)$. By Theorem 1, the time complexity meets the lower bound of the problem and is therefore optimal. □

4. Concluding remarks

In this paper, we give an optimal algorithm for the maximum-density path problem in a tree. Our algorithm uses spine decomposition and a previous algorithm for the maximum density segment problem. The reason that spine decomposition is better than centroid decomposition is that some results obtained in the subtrees can be utilized at the merging step. By centroid decomposition, there is no obvious way to do this. Hopefully spine decomposition may find more applications in other tree problems. For example, the algorithm in this paper can be easily modified to solve the maximum-sum path problem in a tree with the same time complexity $O(n \log n)$. For that problem, an optimal $O(n \log n)$ algorithm was recently also shown by Liu and Chao [11] with a different approach from ours.

Acknowledgements

The author would like to thank the anonymous referees. Their suggestions improve the representation of this article very much. Thanks also to H.-F. Liu and K.-M. Chao for providing their preprint paper [11]. The author was supported in part by an NSC Grant NSC97-2221-E-366-001-MY3 from the National Science Council, Taiwan.

References

[1] M. Ben-Or, Lower bounds for algebraic computation trees, in: Proceedings of the 15th Annual ACM Symposium on Theory of Computing, 1983, pp. 80–86.
[2] R. Benkoczi, Cardinality constrained facility location problems in trees, PhD thesis, School of Computing Science, Simon Fraser University, Burnaby, BC, Canada, May 2004.
[3] R. Benkoczi, B.K. Bhattacharya, D. Breton, Efficient computation of 2-medians in a tree network with positive/negative weights, Discrete Math. 306 (14) (2006) 1505–1516.
[4] K.-M. Chung, H.-I. Lu, An optimal algorithm for the maximum density segment problem, SIAM J. Comput. 34 (2) (2004) 373–387.
[5] R. Cole, U. Vishkin, The accelerated centroid decomposition technique for optimal parallel tree evaluation in logarithmic time, Algorithmica 3 (1988) 329–346.
[6] M.H. Goldwasser, M.-Y. Kao, H.-I. Lu, Linear-time algorithms for computing maximum-density sequence segments with bioinformatics applications, J. Comput. System Sci. 70 (2) (2005) 128–144.
[7] S.-Y. Hsieh, C.-S. Cheng, Finding a maximum-density path in a tree under the weight and length constraints, Inform. Process. Lett. 105 (2008) 202–205.
[8] H.C. Lau, T.H. Ngo, B.N. Nguyen, Finding a length-constrained maximum-sum or maximum-density subtree and its application to logistics, Discrete Optimization 3 (4) (2006) 385–391.
[9] R.-R. Lin, W.-H. Kuo, K.-M. Chao, Finding a length-constrained maximum-density path in a tree, J. Comb. Optim. 9 (2) (2005) 147–156.
[10] Y.-L. Lin, T. Jiang, K.-M. Chao, Algorithms for locating the length-constrained heaviest segments with applications to biomolecular sequence analysis, J. Comput. System Sci. 65 (2002) 570–586.
[11] H.-F. Liu, K.-M. Chao, Algorithms for finding the weight-constrained k longest paths in a tree and the length-constrained k maximum-sum segments of a sequence, Theor. Comput. Sci. 407 (1–3) (2008) 349–358.
[12] A. Tamir, An $O(pn^2)$ algorithm for the p-median and related problems on tree graphs, Oper. Res. Lett. 19 (1996) 59–64.
[13] B.Y. Wu, K.-M. Chao, C.-Y. Tang, An efficient algorithm for the length-constrained heaviest path problem on a tree, Inform. Process. Lett. 69 (2) (1999) 63–67.